

As you can see, PostgreSQL warns you that it cannot delete a role if the role does not exist.



You cannot break PostgreSQL! PostgreSQL will protect itself from your mistakes and it does a very good job of keeping your data safe! The preceding example about the deletion of a nonexistent role is an example of how PostgreSQL protects itself from your mistakes in order to ensure a service that is always stable.

The `DROP ROLE` statement supports the `IF EXIST` clause, which stops PostgreSQL from complaining about the deletion of a role that is missing:

```
postgres=# DROP ROLE IF EXIST this_role_does_not_exist;  
NOTICE:  role "this_role_does_not_exist" does not exist, skipping  
DROP ROLE
```

As you can see, this time PostgreSQL does not raise an error; instead, it displays a notice about the fact that the role does not exist. However, it executes the statement, doing nothing, but reporting success instead of failure. Why could this be useful? Imagine that you have an automated task that is in charge of deleting several roles: if the `DROP ROLE` reports a failure, your task could be interrupted, while with `IF EXIST`, you will rest assured that PostgreSQL will not cause an abort due to a missing role.



There are several statements that support the `IF EXIST` clause, as you will see in later chapters. The idea is to avoid reporting an error when you are not interested in catching it, and you should use, whenever possible, this clause in automating programs.

What happens if you drop a group? Member roles will stay in place, but of course, the association with the group will be lost (since the group has been deleted). In other words, deleting a group does not cascade to its members.

Inspecting existing roles

Now that you know how to create and delete roles, how can you inspect existing roles, including yours? There are different ways to get information about existing roles, and all rely on the PostgreSQL catalogs, the only source of introspection into the cluster.

In order to get information about what role you are running, use the special keyword `CURRENT_ROLE`: you can query it via a `SELECT` statement (such statements will be presented in later chapters, so for now, just blindly use it as shown here):

```
postgres=# SELECT current_role;
 current_role
-----
 postgres
(1 row)
```

If you connect to the database with another user, you will see different results:

```
$ psql -U luca postgres
psql (16.0)
Type "help" for help.

postgres=> SELECT current_role;
 current_role
-----
 luca
(1 row)
```

Knowing your own role is important, but getting information about existing roles and their properties can be even more illuminating. `psql` provides the special `\du` (describe users) command to list all the available roles within the system:

```
$ psql -U postgres
psql (16.0)
Type "help" for help.

postgres=# \du
```

Role name	Attributes
	List of roles

```

-----+-----
--
book_authors | Cannot login
enrico       |
forum        |
forum_admins | Cannot login
forum_emails | No inheritance, Cannot login
forum_stats  | No inheritance, Cannot login
luca         | 1 connection
postgres     | Superuser, Create role, Create DB, Replication, Bypass RLS

```

The `Attributes` column shows the options and properties of the role, many of which will be discussed in *Chapter 10, Users, Roles, and Database Security*. With regard to the login properties, if a role is prevented from connecting interactively to the cluster, a `Cannot login` message will be displayed in the `book_authors` line, like in the preceding example.



The `psql` special command `\drg` will show you all the groups a role is member of.

You can get information about a specific role by directly querying the `pg_roles` catalog, a catalog that contains information about all PostgreSQL roles. For example, to get the basic connection information for the `luca` role, you can execute the following query:

```

postgres=# SELECT rolname, rolcanlogin,
               rolconlimit, rolpassword
             FROM pg_roles
             WHERE rolname = 'luca';

-[ RECORD 1 ]--+-
rolname       | luca
rolcanlogin   | t
rolconlimit   | 1
rolpassword   | *****

```

As you can see, the password is not displayed for security reasons, even if the cluster superuser is asking for it. It is not possible to get the password in plain text: as we've already seen, the passwords are always stored encrypted.

The special catalog `pg_authid` represents the backbone for the `pg_roles` information, and can be queried with the very same statement, but reports the user password (as encrypted text).

The following code shows the result of querying `pg_authid` for the very same user as in the fourth listing; note how the `rolpassword` field contains some more useful information this time:

```
postgres=# SELECT rolname, rolcanlogin, rolconnlimit, rolpassword
           FROM pg_authid WHERE rolname = 'luca';
-[ RECORD 1 ]--+-----
rolname      | luca
rolcanlogin  | t
rolconnlimit | 1
rolpassword  | SCRAM-SHA-256$4096:EC42FTKy6bi/hfslsa4Sw=
```

The password is represented as a hash and the initial part specifies the encryption algorithm used, which nowadays defaults to SCRAM-SHA-256. It is worth noting that, while `pg_roles` can be queried by either superusers and normal users, `pg_authid` can be queried only by superusers.

Managing incoming connections at the role level

When a new connection is established to a cluster, PostgreSQL validates the incoming request at the role level. The fact that the role has the `LOGIN` property is not enough for it to open a new connection to any database within the cluster. This is because PostgreSQL checks the incoming connection request against a kind of firewall table, formerly known as **host-based access**, that is defined within the `pg_hba.conf` file.

If the table states that the role can open the connection to the specified database, the connection is granted (assuming it has the `LOGIN` property); otherwise, it is rejected.

Every time you modify the `pg_hba.conf` file, you need to instruct the cluster to reload the new rules via a HUP signal or by means of a `reload` command in `pg_ctl`.

Therefore, the usual workflow when dealing with `pg_hba.conf` is similar to the following:

```
$ $EDITOR $PGDATA/pg_hba.conf
... modify the file as you wish ...

$ sudo -u postgres pg_ctl reload -D $PGDATA
server signaled
```



In the previous code example \$EDITOR is used to launch the preferred editor, if it has been set. You can set your EDITOR environment variable in many shells by typing `export EDITOR=/bin/vim` (or the path to your preferred editor).

In the Docker images provided for this book, the PGDATA variable is already set. Moreover, the interactive shell is already launched with the user postgres. Therefore, in order to reload the cluster configuration, you needn't worry about EDITOR, PGDATA, nor sudo and can simply write `pg_ctl reload` at the shell prompt.

It is worth noting that a superuser role can instrument the cluster to reload the configuration by means of an SQL statement. Calling the special function `pg_reload_conf()` will perform the same action as issuing a reload to `pg_ctl`:

```
postgres=# SELECT pg_reload_conf();
pg_reload_conf
-----
t
```

The syntax of pg_hba.conf

The `pg_hba.conf` file contains the firewall for incoming connections. Every line within the file has the following structure:

```
<connection-type> <database> <role> <remote-machine> <auth-method>
```

Every part of the line has the following meaning:

- `connection-type` is the type of connection supported by PostgreSQL and is either `local` (meaning via operating system sockets), `host` (TCP/IP connection, either encrypted or not), or `hostssl` (TCP/IP encrypted only connection), or `nohostssl` (TCP/IP non-encrypted connections).
- `database` is the name of a specific database that the line refers to or the special keyword `all`, which means every available database. The special replication keyword is used to handle a special type of connection used to replicate the data to another cluster, and it will be explained in later chapters.
- `role` is the specific role (either a username or a group) that the line refers to or the special keyword `all`, which means all available roles (and groups).

- `remote-machine` is the hostname, IP address, or subnet from which the connection is expected. The special keyword `all` matches with any remote machine that the connection is established from, while the special keywords `samehost` and `samenet` match any hostname or subnet the cluster is attached to.
- `auth-method` dictates how the connection must be handled; more generally, it deals with how the login credentials have to be checked. The main methods are `scram-sha-256`, `md5` (the method used in older versions), `reject` to always refuse the connection, and `trust` to always accept the connection without any regard to supplied credentials.



You cannot name a database or a user with one of the special keywords, e.g., `replication`.

In order to better understand how the system works, the following is an excerpt of a possible `pg_hba.conf` file:

```
host      all          luca      carmensita      scram-sha-256
hostssl   all          test      192.168.222.1/32 scram-sha-256
host      digikamdb    pgwatch2  192.168.222.4/32 trust
host      digikamdb    enrico    carmensita      reject
```

The first line indicates that the user `luca` can connect to every database within the cluster (via the `all` clause) via a TCP/IP connection (via the `host` clause) coming from a host named `carmensita`, but he must provide a valid username/password to verify the SCRAM authentication method.

The second line states that the user `test` can connect to every database in the system over an SSL-encrypted connection (see the `hostssl` clause), but only from a machine that has the IPv4 address of `192.168.222.1`; again, the credentials must pass the SCRAM authentication method.

The third line states that access to the `digikamdb` database is granted only to the `pgwatch2` user over a nonencrypted connection from the host `192.168.222.4`; this time, access is granted (`trust`) without any credential being required.

Finally, the last line rejects any incoming connection from the host named `carmensita`, opened by the user `enrico` against `digikamdb`; in other words, `enrico` is not able to connect to `digikamdb` from the `carmensita` host.



The authentication method `trust` should never be used; it allows any role to connect to the database if the **Host-Based-Access (HBA)** has a rule that matches the incoming connection. This is the method that is used when the cluster is initialized in order to enable the freshly created superuser to connect to the cluster. You can always use this trick as a last resort if you get yourself locked out of your own cluster.

Order of rules in `pg_hba.conf`

The order by which the rules are listed in the `pg_hba.conf` file matters. The first rule that satisfies the logic is applied, and the others are skipped. In order to better understand this, imagine that we want to allow `luca` to connect to any database in the cluster except `forumdb`. The following does not make this happen:

```
host all      luca all scram-sha-256
host forumdb luca all reject
```

Why does the preceding code not work?

Imagine that the user `luca` tries to open a connection to the `forumdb` database: the machine from which the connection is attempted is matched against the `all` keyword with the line containing `luca`, and then the database name is matched against the `all` keyword for the database field.

Since both the remote machine and the database name are subsets of `all`, the connection is passed through the SCRAM-256 authentication method; if the user succeeds in the authentication, the connection is opened. The `reject` line is therefore skipped because the first line matches. On the other hand, exchanging the order of the rules as shown in the following code does work:

```
host forumdb luca all reject
host all      luca all scram-sha-256
```

In this way, when `luca` tries to connect to a database, he gets rejected if the database is `forumdb`; otherwise, he can connect (if he passes the required authentication method).

Merging multiple rules into a single one

One line declares at least one rule, but it is possible to merge multiple lines into a single one. In fact, the role, database, and remote-machine fields allow the definition of multiple matches, each one separated by a `,` (comma).

As an example, suppose we want to give access to the `luca` and `enrico` roles (from the same network that the cluster is running into) to the `forumdb` and `learnpgdb` databases so that `pg_hba.conf` looks like the following:

```
host forumdb luca samenet scram-sha-256
host forumdb enrico samenet scram-sha-256
host learnpgdb luca samenet scram-sha-256
host learnpgdb enrico samenet scram-sha-256
```

Since the database and the role fields can list more than one item, the preceding code can be compressed into the following one:

```
host forumdb,learnpgdb luca samenet scram-sha-256
host forumdb,learnpgdb enrico samenet scram-sha-256
```

We can shrink the rules one step further since the machine from which the database connection can be established is literally the same for both rules, and therefore the final code is as follows:

```
host forumdb,learnpgdb luca, enrico samenet scram-sha-256
```

It should now be clear to you that if more rules have the same authentication method and connection protocol, then it is possible to collapse them into an aggregation. This can help you manage the host-based access configuration.

Using groups instead of single roles

The role field in every `pg_hba.conf` rule can be substituted by the name of a group (remember that a group is itself a role); however, in order to make the rule valid for every member of the group, you have to prefix the group name with a `+` (plus) sign.

To better understand this, consider the example of the `book_authors` group, which includes the `luca` member. The following rule will not allow the `luca` role to access the database:

```
host forumdb book_authors all scram-sha-256
```

Even if the user is a member of the `book_authors` role, it will be denied the ability to log in to the database; the cluster host-based access policy requires the `book_authors` role to be exactly matched by a rule, and in the following command, the `luca` role does not match any rule:

```
$ psql -U luca forumdb
psql: error: could not connect to server:
FATAL: no pg_hba.conf entry for host "192.168.222.1", user "luca",
database "forumdb", SSL off
```

On the other hand, if we clearly state that we want to use the `book_authors` role as a group name, and therefore allow all of its members, the connection can be established by any role that is a member of the group, including `luca`. Therefore, we change the rule to the following:

```
host forumdb +book_authors all scram-sha-256
```

This, in turn (bearing in mind the plus sign), makes the connection possible, as shown here:

```
$ psql -U luca forumdb

forumdb=>
```

The `pg_hba.conf` rules, when applied to a group name (that is, with the `+` preceding the role name) include all the direct and indirect members.

What if we want to allow every group member except one to access the database? Remembering that the rule engine stops at the first match, it is possible to place a reject rule before the group acceptance rule. For example, to allow every member of the `book_authors` group to access the database while preventing the single `luca` role from connecting, you can use the following:

```
host forumdb luca          all reject
host forumdb +book_authors all scram-sha-256
```

The first line will prevent the `luca` role from connecting, even if the following one allows every member of the `book_authors` (including `luca`) to connect: the first match wins and so `luca` is locked out of the database.

Using files instead of single roles

The `role` field of a rule can also be specified as a text file, either line- or comma-separated. This is handy when you deal with long usernames or group names, or with lists produced automatically from batch processes.

If you specify the `role` field with an “at” sign prefix (`@`), the name is interpreted as a line-separated text file (as a relative name to the `PGDATA` directory). For instance, in order to reject connections to all the users and groups listed in the `rejected_users.txt` file, while allowing connections to all the usernames and groups specified in the `allowed_users.txt` file, the `pg_hba.conf` file has to look like the following snippet:

```
host forumdb @rejected_users.txt  all reject
host forumdb @allowed_users.txt   all scram-sha-256
```

The following is the content of the `rejected_users.txt` file, followed by the `allowed_users.txt` file:

```
$ sudo cat $PGDATA/rejected_users.txt
luca
enrico

$ sudo cat $PGDATA/allowed_users.txt
+book_authors, postgres
```

As you can see, it is possible to specify the file contents as either a line-separated list or a comma-separated list of usernames. It is also possible to specify which roles to use as a group by placing a `+` sign in front of the role name.

Inspecting `pg_hba.conf` rules

The `pg_hba.conf` file contains the rules applied to the incoming connections, but since this file could be changed manually without making the cluster reload it, how can you be sure of which rules are applied at the moment? PostgreSQL provides a special catalog named `pg_hba_file_rules` that shows which rules have been applied to the cluster.

You can query the catalogs as a normal table and get information about every line of the `pg_hba.conf` file that has been understood and applied to the current running cluster. As an example, in a fresh PostgreSQL installation, you will probably see an output like the following:

```
postgres=# SELECT line_number, type,
                  database, user_name,
                  address, auth_method
                  FROM pg_hba_file_rules;
 line_number | type | database | user_name | address | auth_method
-----+-----+-----+-----+-----+-----
---
          89 | local | {all}    | {all}    |         | trust
          91 | host  | {all}    | {all}    | 127.0.0.1 | trust
          93 | host  | {all}    | {all}    | ::1      | trust
          96 | local | {replication} | {all}    |         | trust
          97 | host  | {replication} | {all}    | 127.0.0.1 | trust
          98 | host  | {replication} | {all}    | ::1      | trust
```